

STRIPED threat library

STRIDE is a good framework for initiating threat modeling. When enhanced with Privacy concerns it's renamed to "*STRIPED*". The acronym stands for:

- **Spoofing**: pretending to be someone else
- **Tampering**: manipulating data you shouldn't be able to
- **Repudiation**: Denying you did something
- **Information Disclosure**: Accessing data you shouldn't have access to
- **Denial of Service**
- **Elevation of Privilege**: Gaining execution rights you shouldn't have
- **Privacy**: Threats to privacy

Suit	Card	Threat Scenario
Spoofing	2	An attacker could take over the port or socket that the server normally uses.
Spoofing	3	An attacker could try one credential after another and there's nothing to slow them down (online or offline)
Spoofing	4	An attacker can anonymously connect, because we expect authentication to be done at a higher level
Spoofing	5	An attacker can confuse a client because there are too many ways to identify a server
Spoofing	6	An attacker can spoof a server because identifiers aren't stored on the client and checked for consistency on re-connection (that is, there's no key persistence)
Spoofing	7	An attacker can connect to a server or peer over a link that isn't authenticated (and encrypted)
Spoofing	8	An attacker could steal credentials stored on the server and reuse them (for example, a key is stored in a world readable file)
Spoofing	9	An attacker who gets a password can reuse it (Use stronger authenticators)
Spoofing	10	An attacker can choose to use weaker or no authentication
Spoofing	J	An attacker could steal credentials stored on the client and reuse them
Spoofing	Q	An attacker could go after the way credentials are updated or recovered (account recovery doesn't require disclosing the old password)
Spoofing	K	Your system ships with a default admin password, and doesn't force a change
Spoofing	A	You've invented a new Spoofing attack
Spoofing	E	We cannot tell which of our admins edited personal data, as admin accounts are shared.
Tampering	2	An attacker can modify your build system and produce signed builds of your software
Tampering	3	An attacker can take advantage of your custom key exchange or integrity control which you built instead of using standard crypto
Tampering	4	Your code makes access control decisions all over the place, rather than with a security kernel
Tampering	5	An attacker can replay data without detection because your code doesn't provide timestamps or sequence numbers
Tampering	6	An attacker can write to a data store your code relies on
Tampering	7	An attacker can bypass permissions because you don't make names canonical before checking access permissions
Tampering	8	An attacker can manipulate data because there's no integrity protection for data on the network
Tampering	9	An attacker can provide or control state information
Tampering	10	An attacker can alter information in a data store because it has weak/open permissions or includes a group which is equivalent to everyone ("anyone with a Facebook account")
Tampering	J	An attacker can write to some resource because permissions are granted to the world or there are no ACLs
Tampering	Q	An attacker can change parameters over a trust boundary and after validation (for example, important parameters in a hidden field in HTML, or passing a pointer to critical memory)
Tampering	K	An attacker can load code inside your process via an extension point
Tampering	A	You've invented a new Tampering attack
Tampering	E	Data in the database can be 'fixed' by the admins, and nobody will ever know.
Repudiation	2	An attacker can pass data through the log to attack a log reader, and there's no documentation of what sorts of validation are done
Repudiation	3	A low privilege attacker can read interesting security information in the logs
Repudiation	4	An attacker can alter digital signatures because the digital signature system you're implementing is weak, or uses MACs where it should use a signature
Repudiation	5	An attacker can alter log messages on a network because they lack strong integrity controls

Suit	Card	Threat Scenario
Repudiation	6	An attacker can create a log entry without a timestamp (or no log entry is timestamped)
Repudiation	7	An attacker can make the logs wrap around and lose data
Repudiation	8	An attacker can make a log lose or confuse security information
Repudiation	9	An attacker can use a shared key to authenticate as different principals, confusing the information in the logs
Repudiation	10	An attacker can get arbitrary data into logs from unauthenticated (or weakly authenticated) outsiders without validation
Repudiation	J	An attacker can edit logs and there's no way to tell (perhaps because there's no heartbeat option for the logging system)
Repudiation	Q	An attacker can say "I didn't do that," and you'd have no way to prove them wrong
Repudiation	K	The system has no logs
Repudiation	A	You've invented a new Repudiation attack
Repudiation	E	We don't log personal data access, because we do not process any customer data, only employee data.
Repudiation	F	We log changes and deletions of personal data, but viewing is not logged.
Repudiation	G	We log personal data access, but there is no ongoing monitoring or alerting.
Repudiation	H	Our audit log contains personal data, and we do not record who looks at our audit logs.
Information Disclosure	2	An attacker can brute-force file encryption because there's no defense in place (example defense, password stretching)
Information Disclosure	3	An attacker can see error messages with security sensitive content
Information Disclosure	4	An attacker can read content because messages (say, an email or HTTP cookie) aren't encrypted even if the channel is encrypted
Information Disclosure	5	An attacker may be able to read a document or data because it's encrypted with a non-standard algorithm
Information Disclosure	6	An attacker can read data because it's hidden or occluded (for undo or change tracking) and the user might forget that it's there
Information Disclosure	7	An attacker can act as a 'man in the middle' because you don't authenticate endpoints of a network connection
Information Disclosure	8	An attacker can access information through a search indexer, logger, or other such mechanism
Information Disclosure	9	An attacker can read sensitive information in a file with permissive permissions
Information Disclosure	10	An attacker can read information in files or databases with no access controls
Information Disclosure	J	An attacker can discover the fixed key being used to encrypt
Information Disclosure	Q	An attacker can read the entire channel because the channel (say, HTTP or SMTP) isn't encrypted
Information Disclosure	K	An attacker can read network information because there's no cryptography used
Information Disclosure	A	You've invented a new Information Disclosure attack
Information Disclosure	E	Personal data is being sent over a plaintext connection or email.
Information Disclosure	F	Personal data is being saved on unencrypted media.
Denial of Service	2	An attacker can make your authentication system unusable or unavailable
Denial of Service	3	An attacker can drain our easily replaceable battery (battery, temporary)
Denial of Service	4	An attacker can drain a battery that's hard to replace (sealed in a phone, an implanted medical device, or in a hard to reach location) (battery, persist)
Denial of Service	5	An attacker can spend our cloud budget (budget, persist)
Denial of Service	6	An attacker can make a server unavailable or unusable without ever authenticating but the problem goes away when the attacker stops (server, anonymous, temporary)

Suit	Card	Threat Scenario
Denial of Service	7	An attacker can make a client unavailable or unusable and the problem persists after the attacker goes away (client, auth, persist)
Denial of Service	8	An attacker can make a server unavailable or unusable and the problem persists after the attacker goes away (server, auth, persist)
Denial of Service	9	An attacker can make a client unavailable or unusable without ever authenticating and the problem persists after the attacker goes away (client, anon, persist)
Denial of Service	10	An attacker can make a server unavailable or unusable without ever authenticating and the problem persists after the attacker goes away (server, anon, persist)
Denial of Service	J	An attacker can cause the logging subsystem to stop working
Denial of Service	Q	An attacker can amplify a Denial of Service attack through this component with amplification on the order of 10 to 1
Denial of Service	K	An attacker can amplify a Denial of Service attack through this component with amplification on the order of 100 to 1
Denial of Service	A	You've invented a new Denial of Service attack
Elevation of Privilege	2	An attacker has compromised a key technology supplier
Elevation of Privilege	3	An attacker can access the cloud service which manages your devices
Elevation of Privilege	4	An attacker can escape from a container or other sandbox
Elevation of Privilege	5	An attacker can force data through different validation paths which give different results
Elevation of Privilege	6	An attacker could take advantage of permissions you set, but don't use
Elevation of Privilege	7	An attacker can provide a pointer across a trust boundary, rather than data which can be validated
Elevation of Privilege	8	An attacker can enter data that is checked while still under their control and used later on the other side of a trust boundary
Elevation of Privilege	9	There's no reasonable way for a caller to figure out what validation of tainted data you perform before passing it to them
Elevation of Privilege	10	There's no reasonable way for a caller to figure out what security assumptions you make
Elevation of Privilege	J	An attacker can reflect input back to a user, like cross site scripting
Elevation of Privilege	Q	You include user-generated content within your page, possibly including the content of random URLs
Elevation of Privilege	K	An attacker can inject a command that the system will run at a higher privilege level
Elevation of Privilege	A	You've invented a new Elevation of Privilege attack
Inference	2	We use a common identifier across all the systems, and also expose this to third parties.
Inference	3	Our geolocation data is as accurate as possible, even if we really only need to know which city the user is from.
Minimisation	2	We put absolutely everything in the audit log, so we can positively audit all personal data activities.
Minimisation	3	Our testing data is a month-old copy from production. Fake data just does not have the same feel to it.
Retention/Removal	2	Users' file uploads containing personal data are saved to temp files on the front-end.
Retention/Removal	3	All personal data goes into a large pile in the cloud, and going through it to find individual records would cost a fortune in retrieval and outbound data transfer fees.
Retention/Removal	4	We store personal data on disk, even though we only need it temporarily and could just cache it in memory.
Transfer	2	The application uses an API which makes them our data processor, but we don't know whether this is reflected in our API contract.

Suit	Card Threat Scenario	
Transfer	3	We provide an API that ingests personal data, but we do not know whether we are a data processor or a data controller, and it's not defined in our contracts.
Transfer	4	We call an API with personal data, but we do not know where the API is being hosted geographically.
Transfer	5	We export a database dump by writing a CSV file on an FTP site. What happens to the file after it has been downloaded is not our problem.